[3] Training a Tiny Network Can Be NP-Complete

A. Blum and R. L. Rivest — *Neural Networks* (1992)

Training a 3-Node Neural Network Is NP-Complete — Survey: Introduction; RQ4

The paper in one minute. Problem: Does making a neural network extremely small make finding exactly consistent weights easy? **Method:** A polynomial-time reduction from a combinatorial decision problem to whether a particular three-node threshold network fits all labeled examples. **Result:** This training-consistency decision problem is NP-complete. **Why we care:** Neural-network training can be intrinsically difficult in the worst case, even with very few computing units.

1. Define the exact network before discussing hardness

Here "three nodes" means a *two-layer threshold network* with two hidden threshold units and one output threshold unit (not a modern three-layer ReLU MLP). A threshold neuron computes

$$h(x) = \mathbf{1}\{w^\top x \geq b\}, \quad x \in \{0,1\}^n \text{ or another specified input domain.}$$

The hidden layer produces $h_1(x), h_2(x) \in \{0,1\}$, and the output unit computes

$$\hat{y}(x) = \mathbf{1}\{a_1 h_1(x) + a_2 h_2(x) \geq c\}.$$

A-level reminder. The inequality $w^\top x \geq b$ defines a half-space. Each hidden neuron decides which side of a hyperplane an input lies on. Training chooses the hyperplanes and output rule jointly.

2. The precise decision problem

Given a finite labeled training set $S = \{(x_i, y_i)\}_{i=1}^m$, ask whether there exist thresholds and weights such that

$$\forall i \in \{1, \ldots, m\}, \qquad \hat{y}(x_i) = y_i.$$

This is an *existence-of-perfect-fit* decision problem. Equivalently, for the zero–one empirical error

$$R(\theta) = \sum_{i=1}^{m} \mathbf{1}\{\hat{y}_\theta(x_i) \neq y_i\},$$

we ask whether $\min_\theta R(\theta) = 0$. The equation here restates the decision problem; it is not a differentiable loss suitable for ordinary backpropagation.

3. Why a single perceptron is different

For a *single* linear threshold unit, finding a consistent separator can be formulated as linear inequalities, e.g. for labels $s_i \in \{-1, +1\}$,

$$s_i(w^\top x_i - b) \geq 1 \qquad \text{for every } i,$$

when a strictly separating solution exists and can be rescaled. Linear-programming feasibility is polynomial-time solvable. Adding just a small hidden layer changes the logical structure: the learner must decide *which hidden-unit regions* should account for each sample. The combination creates a combinatorial search problem.

4. What the hardness proof establishes

The authors encode combinatorial consistency constraints into examples for their specific threshold architecture and prove NP-completeness of perfect fitting. They also discuss other simple networks and the dependence of difficulty on representation. The proof does *not* say that arbitrary SGD runs on differentiable ReLU/tanh networks are NP-complete, nor that typical training examples exhibit the constructed worst-case difficulty.

Scope matters. The paper itself explicitly leaves extension to nonlinear units such as sigmoids as an open question. It proves hardness for *threshold* neurons and exact training consistency, not for every activation, loss, or approximate optimization target.

5. Why the result matters to our survey

RQ4: Mathematical guarantees about benign optimization must be read with architectural, data and loss assumptions attached. Worst-case hardness of a tiny threshold network can coexist with efficient convergence theorems for highly overparameterized, specially initialized smooth or ReLU networks.

Our takeaway. A tiny neural network is not automatically easy to train. This is a worst-case existence result, so it motivates the rest of our survey rather than contradicting empirical success of modern optimization.

Sources: Journal DOI https://doi.org/10.1016/S0893-6080(05)80010-3; author-hosted journal reprint https://people.csail.mit.edu/rivest/pubs/BR93.pdf.